Nayi Disha (WasteLoop)

Technical Report — Implementation Architecture, Data Model & Verified Prototype Walkthrough

Central New Delhi demo region · Branch `feature/prototype`
Generated August 19, 2026

SECTION 1

Executive Summary

Nayi Disha (WasteLoop) is an AI+IoT municipal waste management platform. This report documents the **as-built prototype** — a complete, runnable TypeScript/Postgres backend and a fully redesigned React frontend implementing the citizen scan → token → triple-lock → throw → reward loop, OSRM-backed collection-route optimization, and a municipal command center — extending the original architecture in `dustbin_arch.md`.

✅ Fully implemented & verified

- Accounts, JWT + bcrypt auth, role-gated admin routes
- AI classification (deterministic mock behind a swappable interface)
- Time-limited disposal tokens/QR + triple-lock verification (GPS + bin QR + token)
- WasteCoin ledger, badges, redemption catalog
- Real OSRM road-network routing + nearest-neighbor/2-opt multi-stop optimization
- Admin ward/fleet analytics dashboard
- Full pastel teal/blue-green UI with saffron/navy accents, across every page

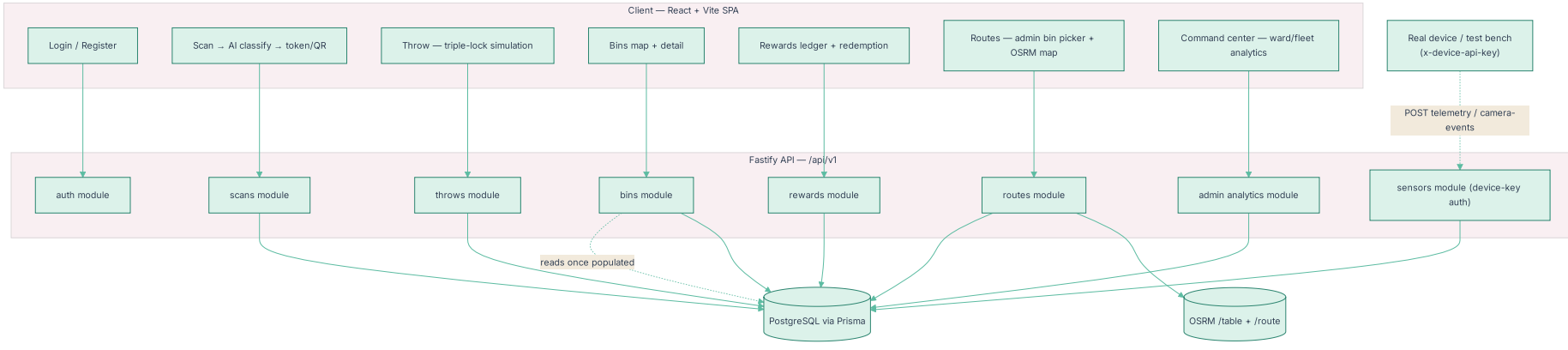
🔶 Intentionally deferred

- Sensor/camera telemetry is **seeded blank** — no fake fill-level or camera data — pending a real device/test-bench integration against the ingestion endpoints
- Route optimizer's fill-threshold auto-select exists in the backend but the UI currently uses explicit bin selection, since fill data isn't live yet
- Real ESP32/Raspberry Pi hardware wiring (endpoints are drop-in ready — see §6)

SECTION 2

Implementation Architecture

The diagram below shows how the running system maps onto the phases described in the original architecture document — client, backend modules, database, and the routing/device-ingestion boundary.

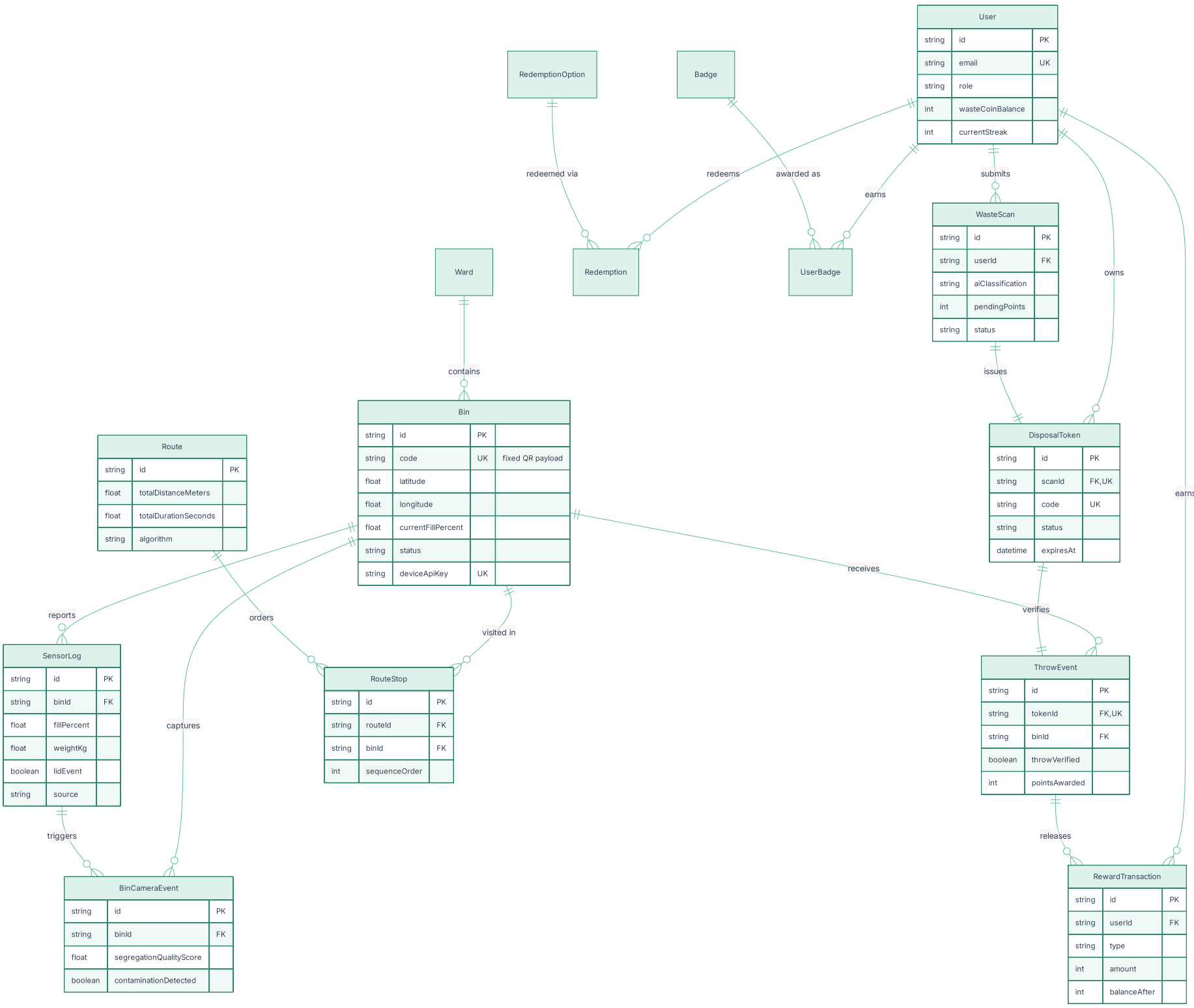


The **sensors module** is authenticated separately from citizen/admin traffic (per-bin `x-device-api-key` header vs. JWT bearer), so a real ESP32 or a test bench calls the exact same endpoints with no code changes required later.

SECTION 3

Database Design

15 Prisma models across four groups: identity/rewards, bins/telemetry, the waste-loop chain (scan → token → throw), and routing.



Hot-path indexes: `SensorLog(binId, recordedAt)`, `BinCameraEvent(binId, createdAt)`, `RewardTransaction(userId, createdAt)`, `DisposalToken(userId, status)`, `Bin(status)` / `Bin(wardId)`.

SECTION 4

API Summary

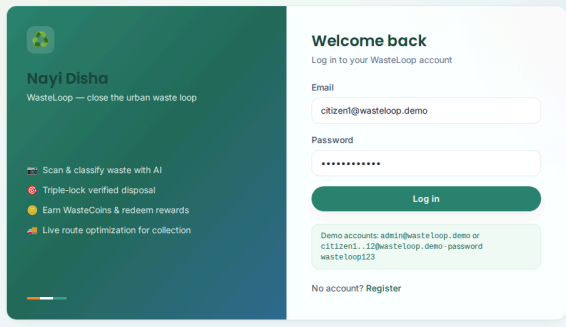
27 routes under `/api/v1`, grouped by module. "Device key" means the per-bin `x-device-api-key` header, not a citizen/admin JWT.

Module	Method	Path	Auth	Description
Auth	POST	`/api/v1/auth/register`	—	Create a citizen account, returns a JWT.
Auth	POST	`/api/v1/auth/login`	—	Log in, returns a JWT.
Auth	GET	`/api/v1/auth/me`	JWT	Current user profile + balances.
Bins	GET	`/api/v1/bins`	JWT	List all bins (map + table source).
Bins	GET	`/api/v1/bins/:id`	JWT	Single bin detail.
Bins	POST	`/api/v1/bins`	JWT (admin)	Create a bin.
Bins	PATCH	`/api/v1/bins/:id`	JWT (admin)	Update a bin (status, label, etc).
Sensors	POST	`/api/v1/bins/:binId/telemetry`	Device key	Ingest fill/weight/lid/motion — device-ready, not user-authenticated.
Sensors	POST	`/api/v1/bins/:binId/camera-events`	Device key	Ingest a bin-camera capture + AI quality score.
Sensors	GET	`/api/v1/bins/:binId/logs`	JWT	Historical sensor log for a bin (BinDetail chart).
Sensors	GET	`/api/v1/bins/:binId/camera-events`	JWT	Historical camera events for a bin.
Scans	POST	`/api/v1/scans`	JWT	Submit photo(s) for AI classification.
Scans	POST	`/api/v1/scans/:id/token`	JWT	Issue a time-limited disposal token/QR.
Scans	GET	`/api/v1/scans`	JWT	List the current user's scans (+ token status).
Scans	GET	`/api/v1/scans/:id`	JWT	Single scan detail.
Throws	POST	`/api/v1/throws`	JWT	Triple-lock verification (GPS + bin QR + token) + reward release.
Rewards	GET	`/api/v1/rewards/transactions`	JWT	WasteCoin ledger for the current user.
Rewards	GET	`/api/v1/rewards/badges`	JWT	All badges + earned status.
Rewards	GET	`/api/v1/rewards/redemption-options`	JWT	Catalog of redeemable rewards.
Rewards	POST	`/api/v1/rewards/redeem`	JWT	Spend WasteCoins on a redemption option.
Routes	POST	`/api/v1/routes/optimize`	JWT (admin)	Optimize a collection route for a set of bin IDs (OSRM + nearest-neighbor/2-opt).
Routes	GET	`/api/v1/routes`	JWT	List recent optimized routes.
Routes	GET	`/api/v1/routes/:id`	JWT	Single route + ordered stops.
Admin	GET	`/api/v1/admin/analytics/ward-summary`	JWT (admin)	Per-ward bin counts + average fill.
Admin	GET	`/api/v1/admin/analytics/segregation-trends`	JWT (admin)	Daily segregation quality/contamination trend from camera events.
Admin	GET	`/api/v1/admin/analytics/bins-status`	JWT (admin)	Bin status breakdown (ACTIVE/FULL/etc).
Admin	POST	`/api/v1/admin/simulate/tick`	JWT (admin)	Advance the software sensor simulator by one tick (dev/demo tool).

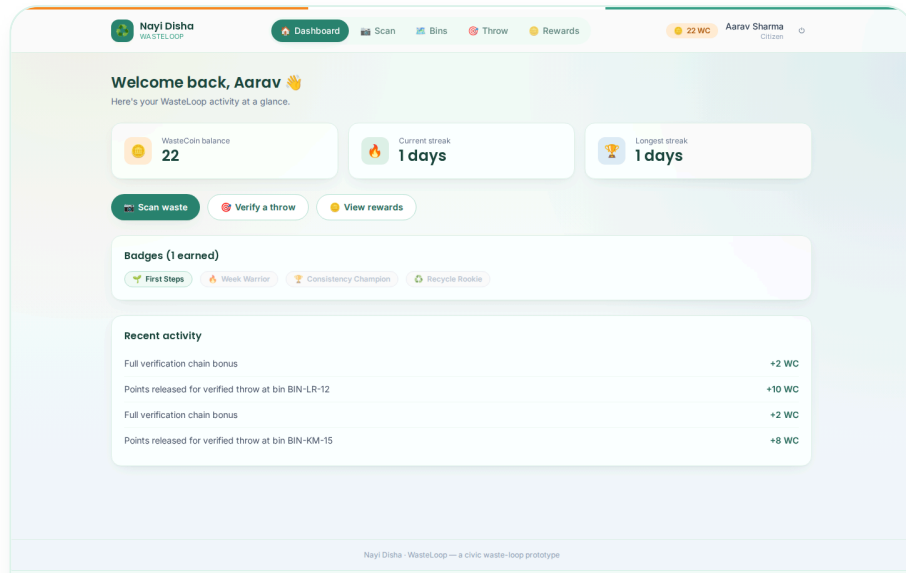
SECTION 5

Verified Demo — Citizen Journey

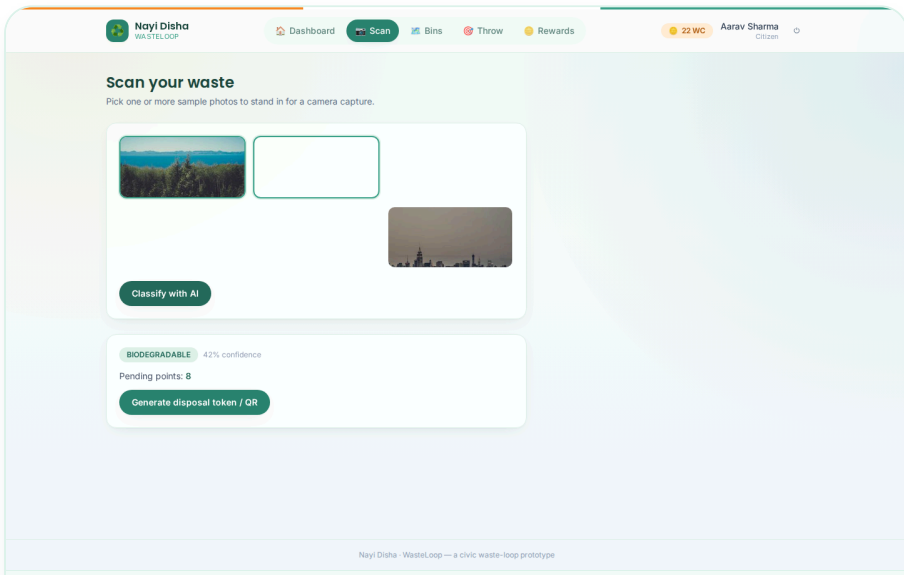
Every screen below was captured against the running app with a live Postgres database and a live OSRM routing backend — not mockups.



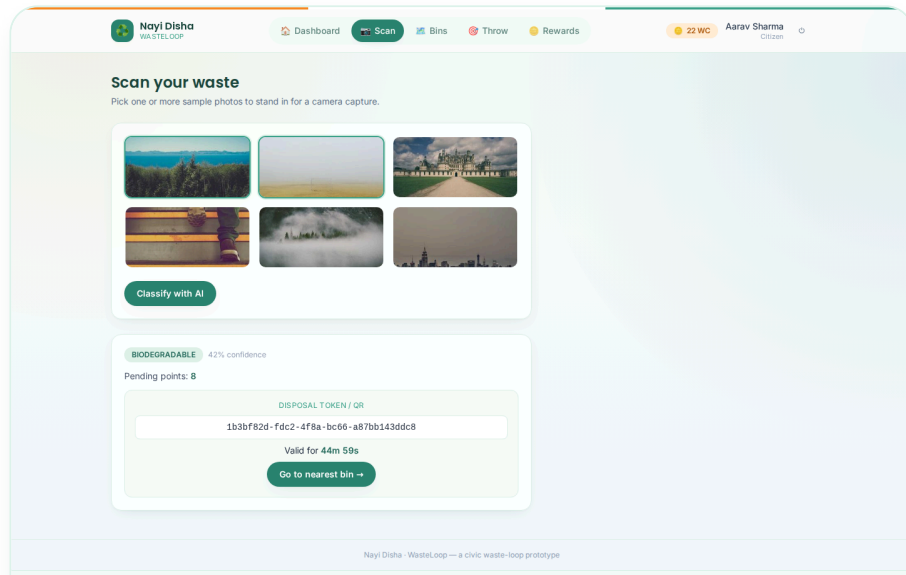
Login — pastel teal/saffron design system



Dashboard — balance, streak, badges, activity



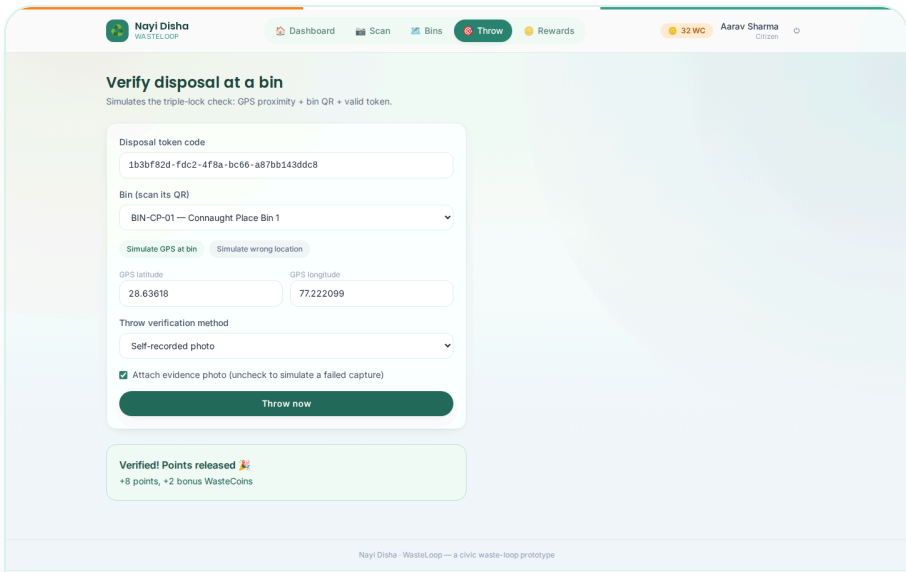
Scan — AI classification result



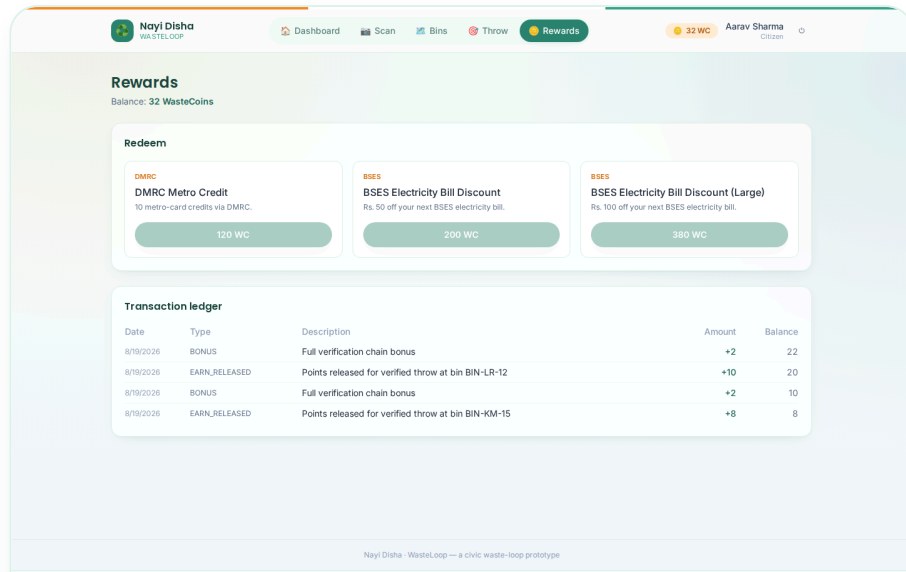
Scan — disposal token/QR issued

SECTION 6

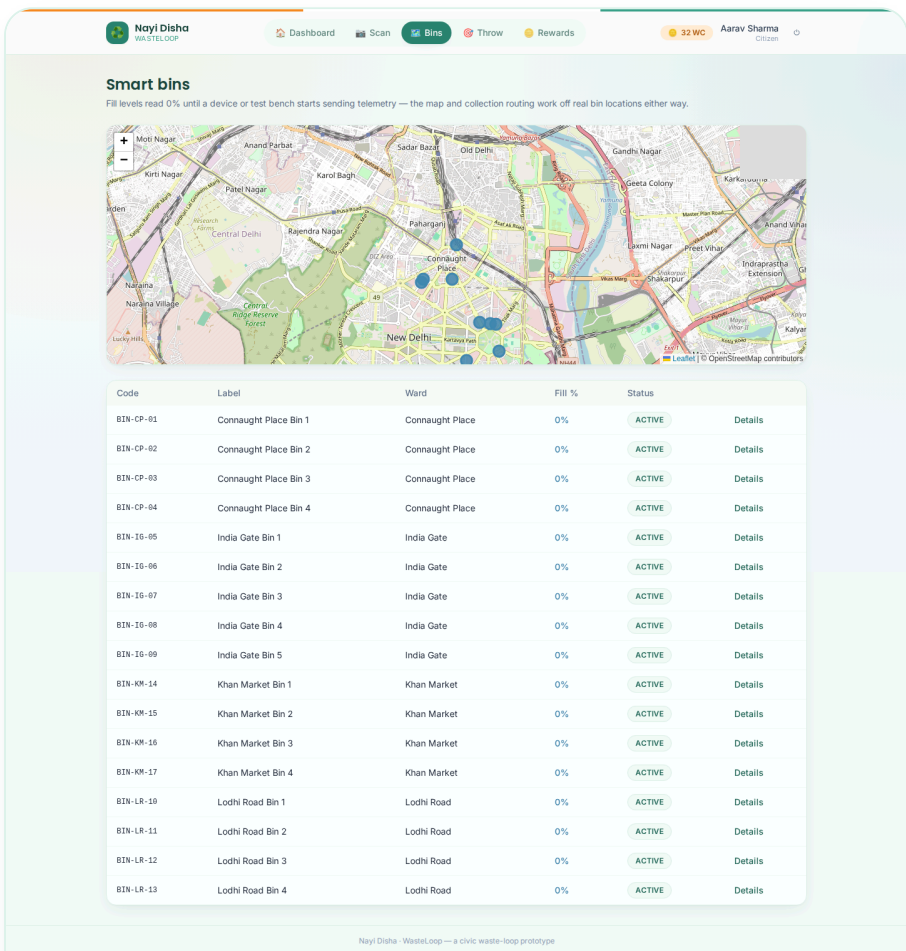
Verified Demo — Throw, Rewards & Bins



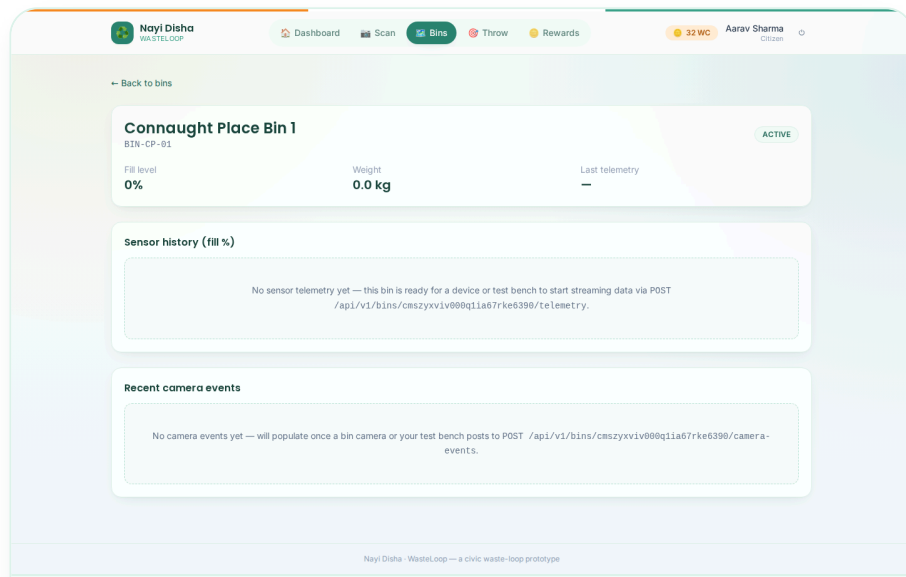
Throw — triple-lock verified, points released



Rewards — ledger + redemption catalog



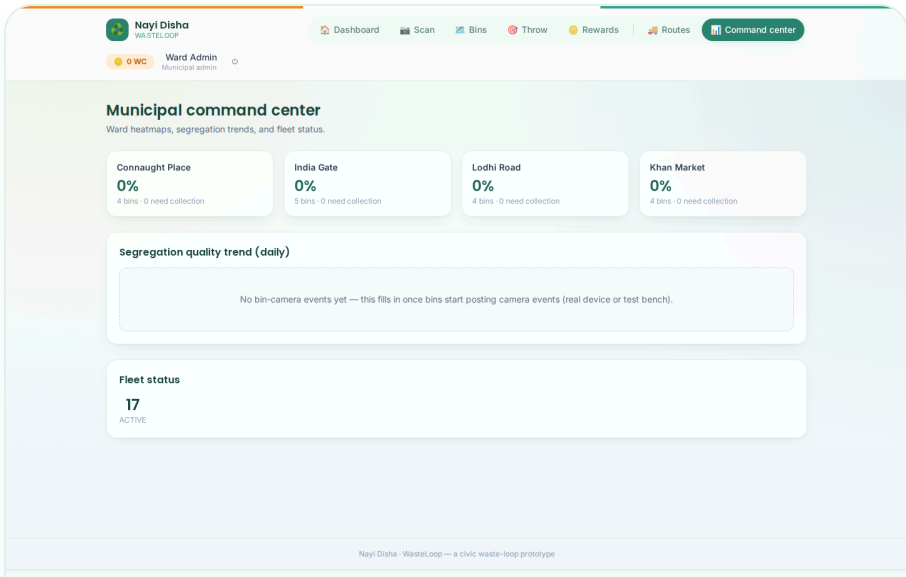
Bins — live map, 0% fill until telemetry starts



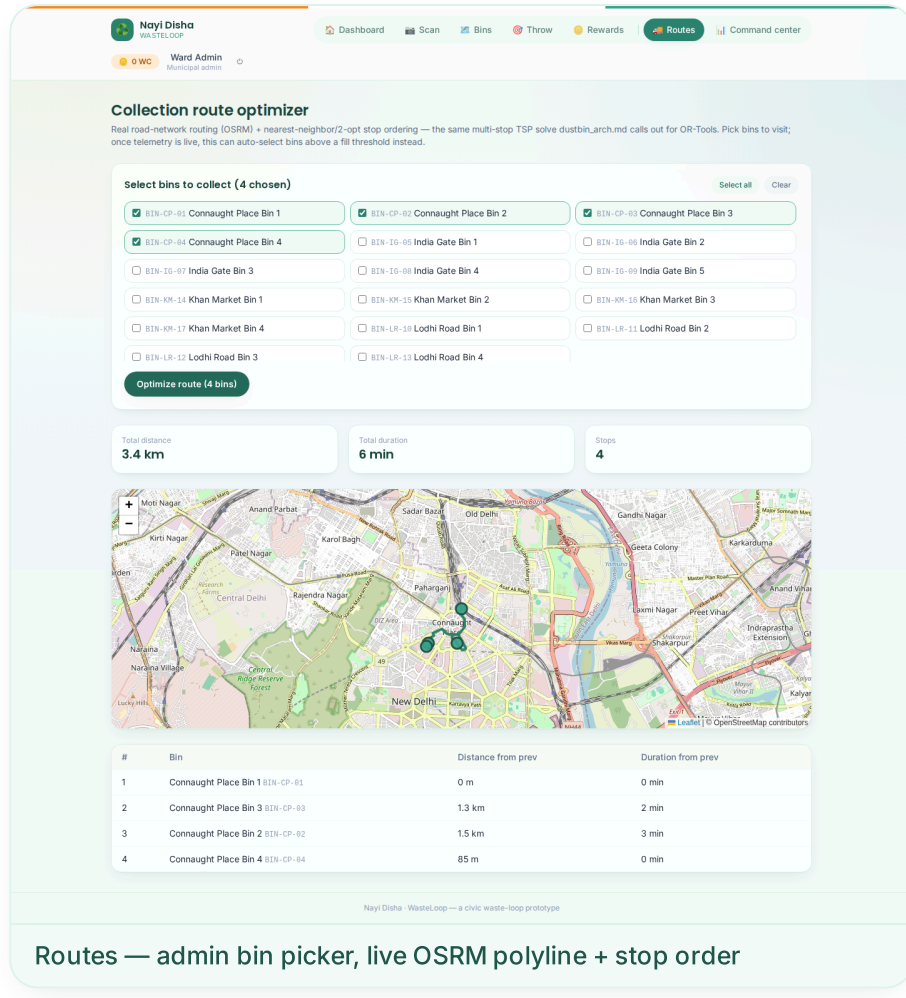
Bin detail — honest empty states, ready for real data

SECTION 7

Verified Demo — Municipal Command Center & Routing



Command center — ward summary + fleet status



Routes — admin bin picker, live OSRM polyline + stop order

SECTION 8

Routing Algorithm

The Routes page lets an admin pick any set of bins (today, manually — once telemetry is live, the backend can auto-select bins above a fill threshold instead, via the same `POST /routes/optimize` endpoint with no body). The optimizer then:

1. Calls OSRM's `/table` service to get a real road-network duration/distance matrix between the selected bins.
2. Runs a nearest-neighbor construction followed by 2-opt local-search improvement over that matrix to find a short visiting order — the same class of solve `dustbin_arch.md` calls out for OR-Tools/TSP.
3. Calls OSRM's `/route` service for the ordered stop sequence to get real road-following geometry, distance, and duration for the final route.
4. Persists the route + ordered stops, and the frontend renders the polyline and stop table.

`OSRM_BASE_URL` defaults to the public OSRM demo server for development and can be pointed at a self-hosted instance (`infra/osrm/`) for anything beyond quick testing.

SECTION 9

Running the Prototype

1. `npm install`
2. `docker compose up -d postgres` (or point `backend/.env` at your own Postgres)
3. `npm run db:migrate --workspace backend && npm run db:seed --workspace backend`
4. `npm run dev:backend` and `npm run dev:frontend`
5. `npm run smoke --workspace backend` to verify the full chain end-to-end

Demo accounts (password `wasteloop123` for all): `admin@wasteloop.demo`, `citizen1@wasteloop.demo` … `citizen12@wasteloop.demo`.

Next step: a real device or test bench authenticates with a bin's `deviceApiKey` via the `x-device-api-key` header and posts to `POST /bins/:binId/telemetry` and `POST /bins/:binId/camera-events` — the exact endpoints this report documents in §4, already live and tested.